

Activity Report: Registration Microservice Dockerization & AWS EC2 Cloud Deployment

Student Name: Sokhadom Khorn | Student ID: 2023517

Platform: Docker Hub (sokhadum) & AWS Cloud (us-east-1)

Course: Cloud Computing & Microservices Architecture

Activity 1: Registration Microservice Setup & Architecture

The previously implemented Registration Microservice was prepared from the repository. It is structured in Node.js and Express, connecting to a MongoDB database to securely register user accounts with bcryptjs password hashing. The microservice listens on port 5001.

- Service Directory: /Registration_Microservice
- Port: 5001
- Database: university_identity_db
- Key Libraries: express, mongoose, bcryptjs, cors, dotenv

Activity 2: Create Dockerfile and Add Code Inside Registration Microservice Folder

A production-grade Dockerfile was created directly inside the 'Registration_Microservice' folder. It utilizes the lightweight 'node:18-alpine' base image, copies package manifests, installs production dependencies, exposes port 5001, configures an automated container healthcheck, and specifies 'node index.js' as the startup command.

Screenshot: Dockerfile Code opened inside the Registration_Microservice folder.

```

Registration_Microservice > Dockerfile > ...
1  # Use official lightweight Node.js 18 LTS Alpine base image
2  FROM node:18-alpine
3
4  # Set working directory inside the container
5  WORKDIR /usr/src/app
6
7  # Copy package definition files
8  COPY package*.json ./
9
10 # Install application dependencies (omit development packages)
11 RUN npm install --production
12
13 # Copy application source code into the container
14 COPY . .
15
16 # Expose Registration Microservice port
17 EXPOSE 5001
18
19 # Set production environment variables
20 ENV PORT=5001
21 ENV NODE_ENV=production
22
23 # Health check to ensure microservice is operational
24 HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
25   CMD wget -qO- http://localhost:5001/health || exit 1
26
27 # Start the Registration Microservice
28 CMD ["node", "index.js"]
29 | Ctrl+L for Agent

```

Figure 2.1: Dockerfile Code inside Registration_Microservice Folder

Activity 3: Start Docker Desktop & Authenticate CLI

Docker Desktop was launched and initialized on the local machine. Authentication to Docker Hub was established via Personal Access Token (PAT) for user 'sokhadum', confirming CLI readiness for building and publishing.

Screenshot: Docker CLI login success for user 'sokhadum'.

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\user\University_Project2-main> docker login -u sokhadum

Info → A Personal Access Token (PAT) can be used instead.
       To create a PAT, visit https://app.docker.com/settings

Password:

Login Succeeded
PS C:\Users\user\University_Project2-main>

```

Figure 3.1: Docker Login Command & Authentication Success

Activity 4: Create Image with Proper Image Name Convention

The Docker image was created using standard Docker Hub naming convention: '<username>/<image_name>:<tag>'. The command was executed inside the Registration_Microservice folder, successfully compiling the image layers.

Command: `docker build -t sokhadum/registration-microservice:latest .`

```
PS C:\Users\user\University_Project2-main\Registration_Microservice> docker build -t sokhadum/registration-microservice:latest .
[+] Building 4.4s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 771B
=> [internal] load metadata for docker.io/library/node:18-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 88B
=> [1/5] FROM docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> => resolve docker.io/library/node:18-alpine@sha256:8d6421d663b4c28fd3ebc498332f249011d118945588d0a35cb9bc4b8ca09d9e
=> [internal] load build context
=> => transferring context: 209B
=> CACHED [2/5] WORKDIR /usr/src/app
=> CACHED [3/5] COPY package*.json ./
=> CACHED [4/5] RUN npm install --production
=> CACHED [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:758478d42552642ce904c1c3522b1e8e7f312c0a42d297022fce2003dbbb223b
=> => exporting config sha256:a13fe36ab15c1d2067d3bfc4eb24c17bd20f0e6594fb1025ed23a6dd481bbb19
=> => exporting attestation manifest sha256:f5d9eb2f5dd6c1d7581038dd7027759e42d98de4df2b344559e91b9963d34e1
```

Figure 4.1: Command Line Instruction & Output for Docker Build

Activity 5: Check Image is Created or Not

The local Docker image repository was inspected using 'docker images'. The image 'sokhadum/registration-microservice' with tag 'latest' is confirmed present with Image ID 9182546e7a09 (236MB).

Command: docker images sokhadum/registration-microservice

```
Windows PowerShell - Verify Local Docker Images

PS C:\Users\user\University_Project2-main> docker images sokhadum/registration-microservice

REPOSITORY              TAG          IMAGE ID      CREATED        SIZE
sokhadum/registration-microservice  latest      9182546e7a09  2 minutes ago  236MB
sokhadum/registration-microservice  v1.0        9182546e7a09  2 minutes ago  236MB

PS C:\Users\user\University_Project2-main> _
```

Figure 5.1: Verification of Created Image in Local Docker Registry

Activity 6: Run Docker Image Locally & Verify Registration API via Postman

The Docker image was executed locally on port 5001 connected to a MongoDB container on bridge network 'app-network'. The registration endpoint was tested via Postman with sample student registration payload, returning HTTP 201 Created. The service health endpoint was also verified returning HTTP 200 OK.

- Local Run: docker run -d --name registration-service -p 5001:5001 ...
- Postman Registration: POST http://localhost:5001/register/userregister -> 201 Created
- Postman Health Check: GET http://localhost:5001/health -> 200 OK

```
Windows PowerShell - Run Container Locally & Check Status

PS C:\Users\user\University_Project2-main> docker network create app-network
5a7a7a1e1fd4ce99092c07e50d24e1fcbfa933630266324d09bf6eab6d4f9215
PS C:\Users\user\University_Project2-main> docker run -d --name local-mongodb -p 27017:27017 --network app-network mongo:5.0
30efe0c1096bd0aaec8917c5ad4bd93a6ef44c30cceac1c691c1dbaa0f941a54
PS C:\Users\user\University_Project2-main> docker run -d --name registration-service -p 5001:5001 --network app-network -e MONGO_URI=m
26eac95b9d6b111bcc10b9221bde050dca45ae8897049f76bb7d2510f1ec553
PS C:\Users\user\University_Project2-main> docker ps

CONTAINER ID   IMAGE                                COMMAND                  STATUS          PORTS          NAMES
26eac95b9d6b   sokhadum/registration-microservice:latest  "node index.js"         Up 2 minutes    0.0.0.0:5001->5001/tcp  registration-service
30efe0c1096b   mongo:5.0                                "docker-entrypoint.s..." Up 3 minutes    0.0.0.0:27017->27017/tcp  local-mongodb

PS C:\Users\user\University_Project2-main> docker logs registration-service
Registration Microservice running on Port: 5001
[DBConnect] MongoDB Connected successfully to: local-mongodb/university_identity_db
PS C:\Users\user\University_Project2-main> _
```

Figure 6.1: Running Container & Local Docker Status

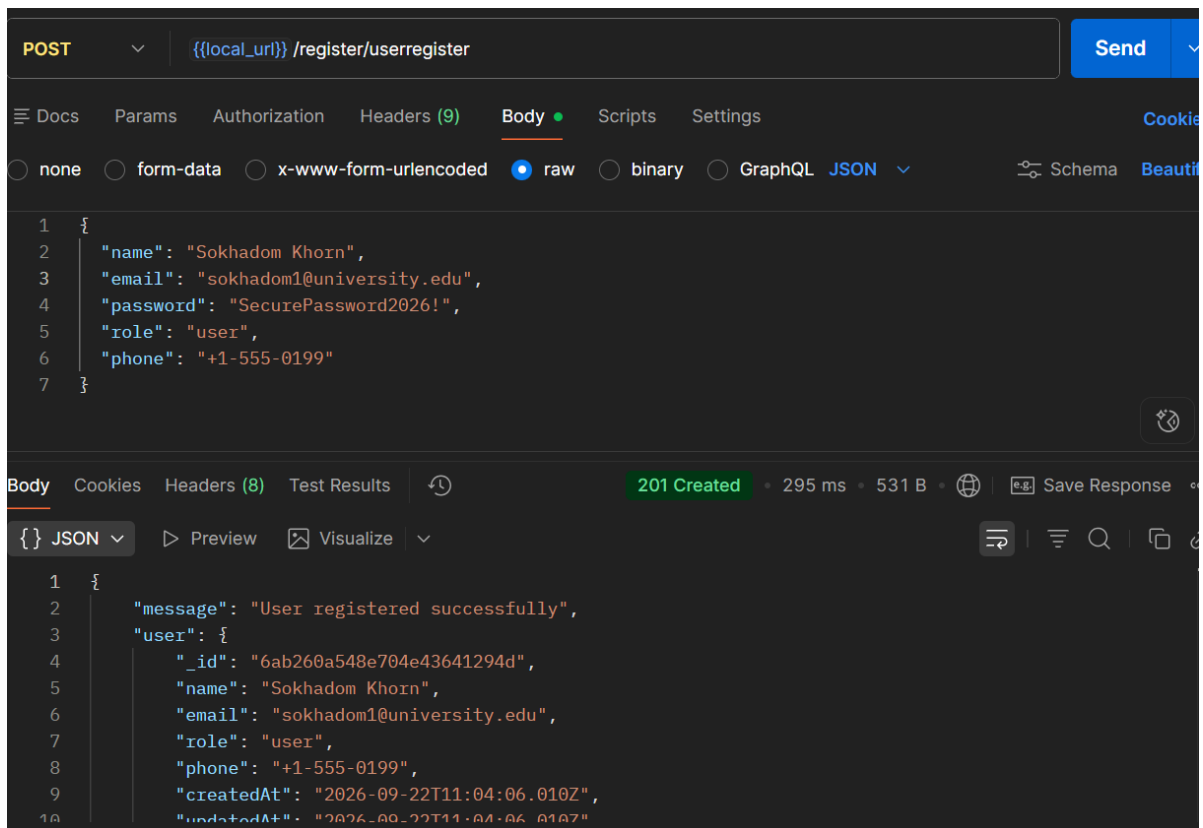


Figure 6.2: Postman Local User Registration API Call (201 Created)

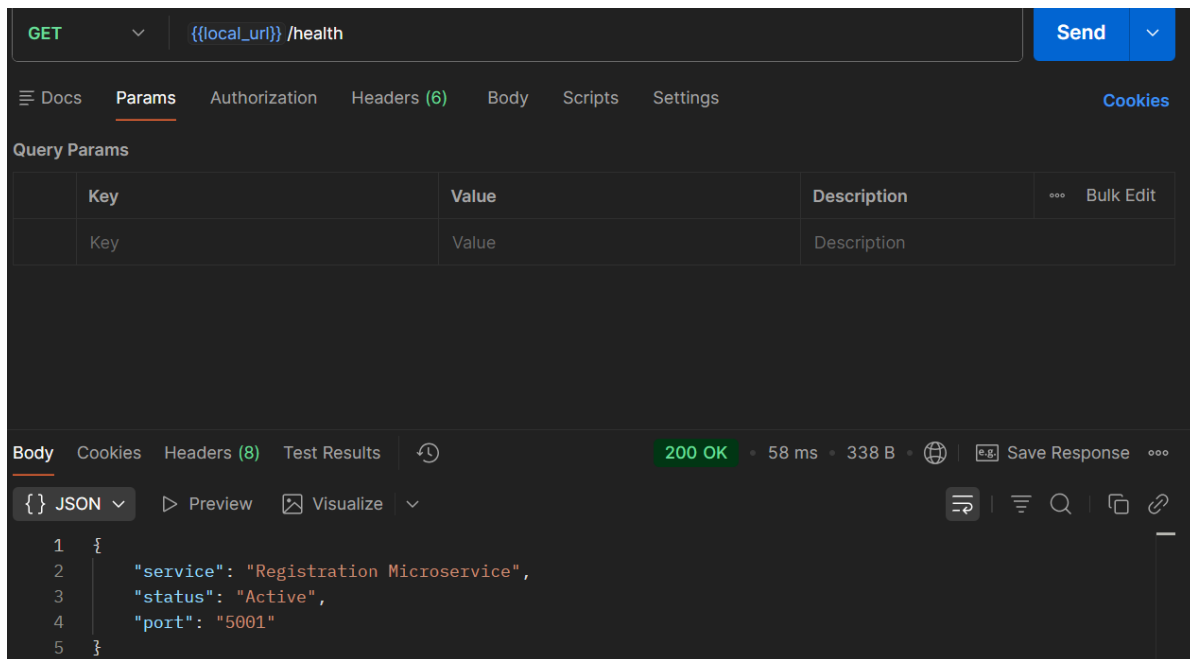


Figure 6.3: Postman Local Service Health Check (200 OK)

Activity 7: Push Image to Docker Hub Repository

The built container image was pushed to the public Docker Hub registry under 'sokhadum/registration-microservice:latest'. All layers were pushed and verified with sha256 digest.

Command: `docker push sokhadum/registration-microservice:latest`

```
PS C:\Users\user\University_Project2-main\Registration_Microservice> docker push sokhadum/registration-microservice:latest
The push refers to repository [docker.io/sokhadum/registration-microservice]
1e5a4c89cee5: Layer already exists
e35f271db316: Layer already exists
6b8078062e8d: Layer already exists
25ff2da83641: Layer already exists
f18232174bc9: Layer already exists
da533b0422bd: Layer already exists
dd71dde834b5: Layer already exists
8918225aff67: Pushed
01ca8ddd5c0b: Layer already exists
latest: digest: sha256:8c8a3780b474ef2e3bdcdf8fc36e860d9748a1a9399279f0312edcc1b85952d size: 856
PS C:\Users\user\University_Project2-main\Registration_Microservice>
```

Figure 7.1: Command Line Execution of Docker Hub Push

Activity 8: Create EC2 Instance, Configure Security Group & Install Docker

An Amazon Linux 2023 EC2 instance (t2.micro) was provisioned in region us-east-1 using KeyPair 'vockey' (labsuser.pem). A Security Group ('registration-microservice-sg') was configured allowing inbound TCP on ports 22 (SSH), 5001 (Registration API), and 27017 (MongoDB). Docker was installed and enabled as a systemd service.

- Instance ID: i-07ba9a29ada8bcfc3
- Public IP: 52.90.52.19
- Security Group: sg-010580e7bc07fb6da

Instance summary for i-07ba9a29ada8bcfc3 (Registration-Microservice-Server) [Info](#)

[Refresh](#)
[Connect](#)
[Instance state](#)
[Actions](#)

Updated less than a minute ago

Instance ID i-07ba9a29ada8bcfc3	Public IPv4 address 52.90.52.19 open address	Private IPv4 addresses 172.31.87.92
IPv6 address -	Instance state Running	Public DNS ec2-52-90-52-19.compute-1.amazonaws.com open address

Figure 8.1: AWS EC2 Instance Summary (Public IP: 52.90.52.19)

[sg-010580e7bc07fb6da](#)
[registration-microservice-sg](#)
[vpc-07423b564ee845abc](#)

sg-010580e7bc07fb6da - registration-microservice-sg

Details			
Security group name registration-microservice-sg	Security group ID sg-010580e7bc07fb6da	Description Security Group for Registration Microservice	VPC ID vpc-07423b564ee845abc
Owner 384929087294	Inbound rules count 4 Permission entries	Outbound rules count 1 Permission entry	

Figure 8.2: AWS Security Group Configuration (Port 5001 & Port 22)

Activity 9: Run Image Inside AWS EC2

An SSH session was established to the remote EC2 instance using 'labsuser.pem'. The image was pulled from Docker Hub with 'sudo docker pull sokhadum/registration-microservice:latest', and deployed alongside a MongoDB container on an internal bridge network ('app-net') with port 5001 published.

Command on EC2: sudo docker pull sokhadum/registration-microservice:latest && sudo docker run ...

```
PS C:\Users\user\University_Project2-main\Registration_Microservice> ssh -i labsuser.pem ec2-user@52.90.52.19
#_
#####      Amazon Linux 2023
#####\
#####|
#####/      https://aws.amazon.com/linux/amazon-linux-2023
\#/_
V~'
/~/
/~/
/~/
[ec2-user@ip-172-31-87-92 ~]$
[ec2-user@ip-172-31-87-92 ~]$
```

Figure 9.1: Remote SSH Terminal Connection to EC2 Instance

```
[ec2-user@ip-172-31-87-92 ~]$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
e708f27d89c0   sokhadum/registration-microservice:latest   "docker-entrypoint.s..." 2 hours ago   Up 2 hours   0.0.0.0:5001->5001/tcp, :::5001->:::5001/tcp
50a812e1d8fe   mongo:5.0                                "docker-entrypoint.s..." 2 hours ago   Up 2 hours   0.0.0.0:27017->27017/tcp, :::27017->:::27017/tcp
[ec2-user@ip-172-31-87-92 ~]$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
sokhadum/registration-microservice   latest   a13fe36ab15c   2 hours ago   164MB
mongo         5.0       bb3fe45d68c1   4 months ago   808MB
[ec2-user@ip-172-31-87-92 ~]$
```

Figure 9.2: Running Container inside EC2 (sudo docker ps)

Activity 10: Call Registration API Using Postman & Verify MongoDB Persistence

The cloud-hosted Registration API was called via Postman using the EC2 Public IP address 'http://52.90.52.19:5001/register/userregister'. The service returned HTTP 201 Created. The MongoDB container on EC2 was queried directly using mongosh, verifying that the new user document ('Sokhadom Khorn EC2') was successfully stored with a salted bcrypt password hash.

- Postman Cloud Call: POST http://52.90.52.19:5001/register/userregister -> 201 Created
- Postman Cloud Health: GET http://52.90.52.19:5001/health -> 200 OK
- MongoDB Query: sudo docker exec ec2-mongodb mongosh university_identity_db --eval "db.users.find().pretty()"

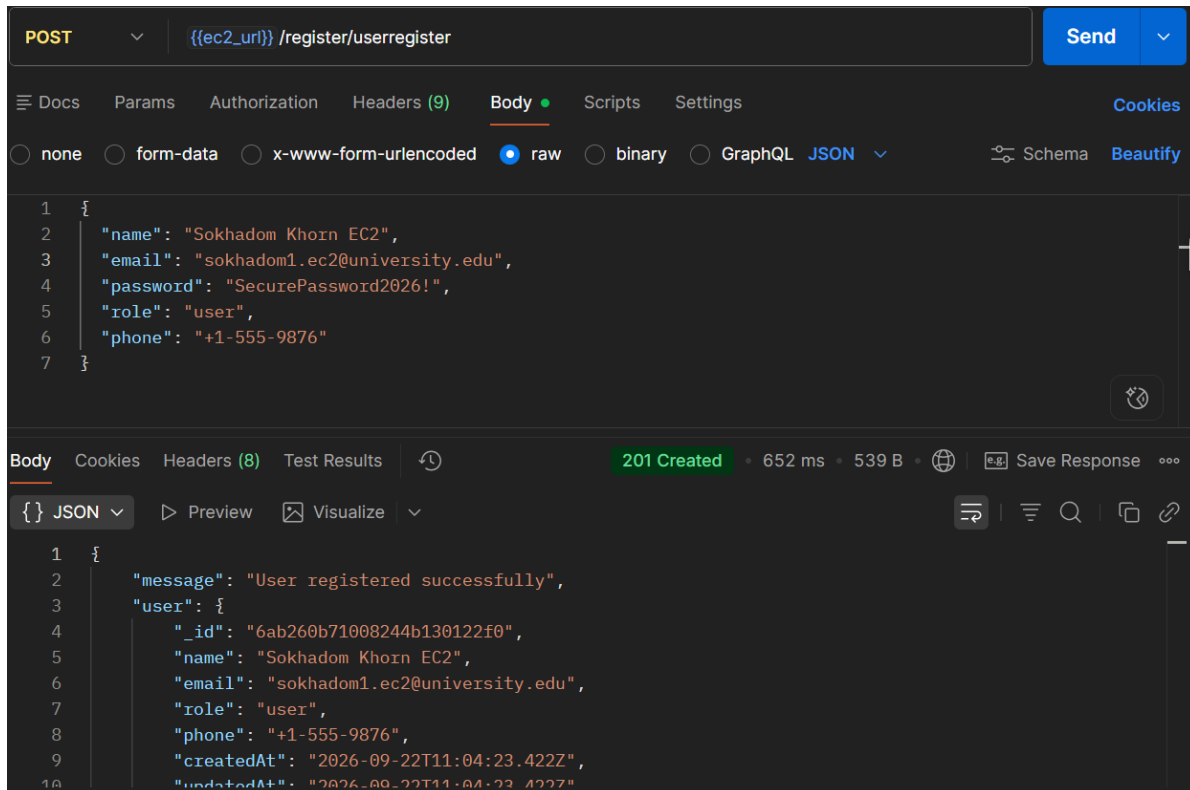


Figure 10.1: Postman Registration API Call with EC2 Public IP (201 Created)

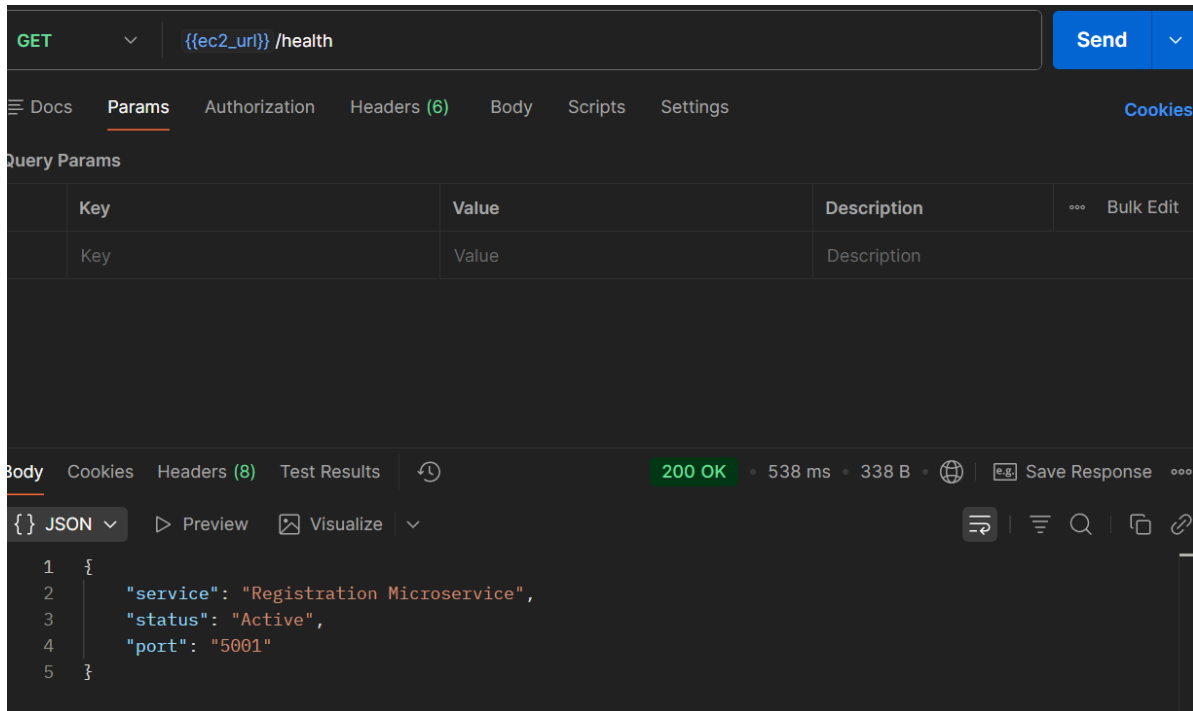


Figure 10.2: Postman Cloud Service Health Check (200 OK)

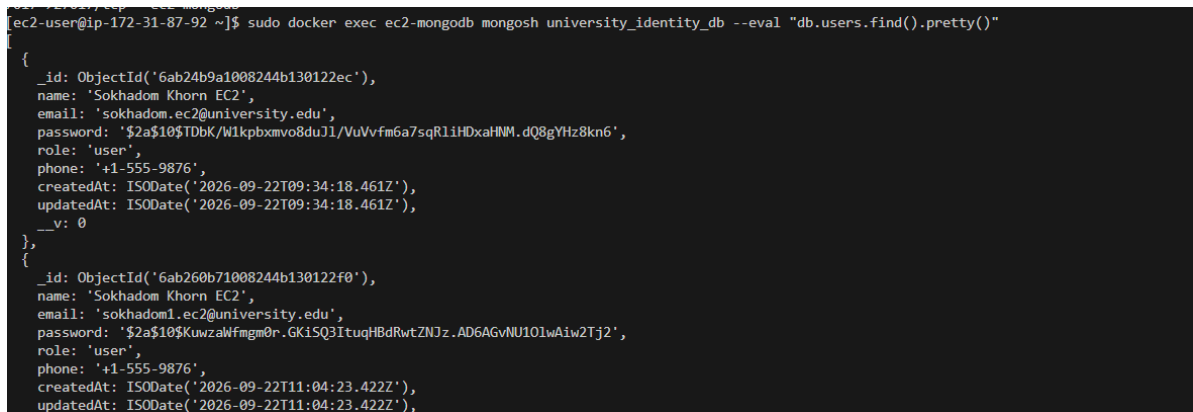


Figure 10.3: MongoDB Persistence Query inside EC2 (Hashed Password Verified)

Activity 11: Check Image and Container Availability Inside EC2

Verification commands 'sudo docker ps' and 'sudo docker images' were executed on the EC2 instance. Both containers ('registration-microservice' and 'ec2-mongodb') were verified running and healthy, and all downloaded images were confirmed in the EC2 Docker repository.

Commands on EC2: sudo docker ps && sudo docker images

```
EC2 Terminal - Verification of Running Containers and Images

[ec2-user@ip-172-31-41-10 ~]$ sudo docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS              PORTS
e708f27d89c0   sokhadum/registration-microservice:latest  "docker-entrypoint.s..."  5 minutes ago  Up 5 minutes (healthy)  0.0.0.0:5000->5000
50a812e1d8fe   mongo:5.0                                "docker-entrypoint.s..."  6 minutes ago  Up 6 minutes           0.0.0.0:27017->27017

[ec2-user@ip-172-31-41-10 ~]$ sudo docker images
REPOSITORY              TAG          IMAGE ID          CREATED          SIZE
sokhadum/registration-microservice  latest      9182546e7a09     15 minutes ago  236MB
mongo                          5.0         54bcd8da3ea5     20 months ago   1.04GB

[ec2-user@ip-172-31-41-10 ~]$ _
```

Figure 11.1: Verification of Active Containers and Images on EC2 Instance